

Recommendation Systems for

Personalized Content Discovery

Technical Report — Netflix Prize Dataset

Prepared by — Anuj Jangir
Rakshit Raj

Dataset Metric	Value
Total Ratings (file 1)	24,053,764
Users (sampled 5%)	23,537
Movies (after filter)	2,586
Matrix Sparsity	98.87%
Mean Rating	3.60 / 5.0

1. Problem Understanding

Every day, billions of users interact with platforms that silently decide what they should watch next. Behind this experience lies a recommendation engine — a system that learns from past behaviour to surface content a user is likely to enjoy. Getting this right is not merely a technical challenge; it directly shapes user retention, engagement, and the overall health of a streaming business.

The Netflix Prize dataset, released in 2006, remains one of the most studied benchmarks in the recommender-systems community. It contains over 100 million explicit ratings from nearly half a million users across roughly 17,000 movies — a scale that makes it both realistic and computationally demanding.

Objective

Our goal is to build a recommendation engine that goes beyond simple rating prediction. The system must learn user preferences from sparse historical interactions, rank unseen items in a personalised order, and produce recommendations that are explainable and practically useful.

Core Challenges

- **Extreme Sparsity (98.87%):** Each user has rated only a tiny fraction of available movies, leaving almost no direct signal for most user-item pairs.
- **Long-tail Distribution:** A handful of popular movies receive thousands of ratings while the majority have fewer than thirty.
- **Implicit Preference Signals:** A rating of 4 or 5 does not just mean the user liked the film — it also reveals genre taste, era preference, and viewing habits.
- **Cold Start:** New users or newly added movies have no interaction history, making personalisation near-impossible without auxiliary strategies.

1. Exploratory Data Analysis

1. Dataset Overview

Metric	Raw (file 1)	After Filtering
Total Ratings	24,053,764	1,088,949
Unique Users	23,537 (5% sample)	13,067
Unique Movies	4,492	2,586
Matrix Sparsity	98.87%	—
Mean Rating	3.60	3.61
Median Rating	4.0	4.0

Table 1 — Key dataset statistics before and after preprocessing.

2.2 Rating Distribution

Ratings follow a pronounced left-skewed distribution. The most common rating is 4, followed by 3 and 5. Ratings of 1 and 2 are comparatively rare, suggesting that users either genuinely enjoyed the content or simply chose not to rate films they disliked — a well-known positivity bias in explicit feedback datasets. The mean (3.60) sitting above the midpoint (3.0) confirms this tendency.

2.3 User Activity Patterns

The distribution of ratings per user is highly skewed. The median user has rated just 24 movies, while the mean is 51.6 — pulled upward by a small cohort of power users (maximum: 2,263 ratings). This long tail is typical of real-world platforms and has a direct implication: most users have very sparse profiles, making personalisation harder.

Key Insight: Roughly 50% of users have rated fewer than 24 movies. Any model must generalise well from limited evidence rather than relying on dense interaction histories.

2.4 Content Popularity Trends

A handful of titles dominate the rating counts. Pirates of the Caribbean: The Curse of the Black Pearl leads with nearly 10,000 ratings in the sampled subset, followed by What Women Want, Bruce Almighty, and The Italian Job. These titles are all from the early 2000s, consistent with the dataset collection period (2000–2005). The long tail on the movie side means that a naive popularity-based baseline would perpetually recommend the same blockbusters, ignoring the preferences of users who gravitate toward niche content.

2.5 Sparsity Analysis

With 13,067 users and 2,586 movies after filtering, the theoretical user-item matrix contains approximately 33.8 million cells. Only around 1.09 million are observed — a fill rate of roughly 3.2% in the filtered subset, still 96.8% sparse. This level of sparsity makes memory-based approaches (like raw nearest-neighbour) unreliable and strongly motivates latent-factor methods that compress interactions into dense low-dimensional representations.

3. Methodology

1. Data Pipeline

- **Loading:** Raw Netflix Prize files parsed line-by-line; movie IDs extracted from colon-terminated headers.
- **Sampling:** 5% of users randomly selected to keep training tractable on a standard machine.
- **Filtering:** Users and movies with fewer than 20 ratings removed (`min_user_ratings=20`, `min_movie_ratings=20`) to reduce noise from extremely sparse interactions.
- **Encoding:** Original user/movie IDs mapped to contiguous 0-based integer indices for matrix construction.
- **Splitting:** Stratified 80/20 train-test split per user, ensuring every user appears in both sets.

3.2 Train-Test Split Strategy

A user-stratified split was used instead of a simple random split. For each user, 80% of their ratings form the training set and 20% the test set. This guarantees that every user in the test set has a known profile in training — a realistic evaluation scenario that avoids artificially penalising models for genuinely cold users.

3.3 Relevance Definition for MAP@10

A movie is considered **relevant** for MAP@10 computation if the user's actual rating is ≥ 3.5 . This threshold was chosen to capture genuinely positive interactions (ratings 4 and 5) while also including borderline-positive ratings (3.5 is not possible in integer ratings but the threshold is inclusive of 4+). In our dataset, approximately 62% of ratings fall at or above this threshold.

3.4 Top-10 Recommendation Procedure

- For each evaluation user, retrieve all movies they rated in the test set.
- Exclude movies the user already rated in the training set (no re-recommendation).
- Score all remaining candidate movies using the model's `predict_rating` function.
- Rank candidates by predicted score descending; return the top 10.
- Compare this ranked list against the user's actual relevant items (rating ≥ 3.5).

4. Model Design

1. SVD — Matrix Factorization (Primary Model)

Singular Value Decomposition factorises the user-item rating matrix R into two low-rank matrices — a user latent factor matrix P and an item latent factor matrix Q — such that $R \approx P \times Q^T$. Each user and item is represented as a dense vector of k latent dimensions ($k=100$ here). The predicted rating for user u on item i is:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^T \cdot q_i$$

where μ is the global mean, b_u and b_i are user/item biases, and p_u, q_i are the latent vectors. Parameters are learned by stochastic gradient descent minimising regularised squared error. We used 100 factors, 20 epochs, learning rate 0.005, and regularisation 0.02.

2. Item-Based Collaborative Filtering

Item-CF computes pairwise cosine similarity between all movie vectors (each movie represented as a sparse vector of user ratings). To predict r_{ui} , the model finds the $k=20$ most similar movies to item i that user u has already rated, then computes a similarity-weighted average of those ratings. The approach is interpretable — 'you liked X and Y, and Z is similar to both' — but scales quadratically with the number of items.

3. ALS — Implicit Alternating Least Squares

ALS treats ratings as confidence-weighted implicit signals rather than explicit preferences. The confidence for a rating r is $c = 1 + \alpha \times r$ ($\alpha=40$). The model alternates between solving for user factors (holding item factors fixed) and item factors (holding user factors fixed), each step being a closed-form least squares problem. While well-suited to click or view data, its performance on explicit rating prediction is weaker since it was not designed to minimise MSE on ratings.

4. Hyperparameter Summary

Model	Key Hyperparameters	Values
SVD	Factors / Epochs / LR / Reg	100 / 20 / 0.005 / 0.02
ItemCF	Neighbours (k)	20
ALS	Factors / Iterations / α	100 / 20 / 40.0

Table 2 — Hyperparameter configurations for all three models.

5. Evaluation Metrics

1. RMSE — Root Mean Squared Error

RMSE measures how accurately the model predicts the actual numeric rating a user would give an item. It penalises large errors more heavily than small ones due to the squaring operation, making it sensitive to outlier predictions.

$$\text{RMSE} = \sqrt{[(1/n) \times \sum_{ui} (r_{ui} - \hat{r}_{ui})^2]}$$

2. MAP@10 — Mean Average Precision at 10

While RMSE tells us how accurate our rating predictions are, MAP@10 tells us how good our ranked recommendation lists are. For each user, Average Precision at 10 is computed as:

$$\text{AP@10} = (1 / \min(|R|, 10)) \times \sum_{k=1..10} P(k) \cdot \text{rel}(k)$$

where $P(k)$ is precision at position k and $\text{rel}(k)$ is 1 if the item at rank k is relevant (rating ≥ 3.5), else 0. MAP@10 is the mean of AP@10 across all evaluation users (capped at 500 for speed). A higher MAP@10 indicates the model is placing relevant items near the top of the recommendation list.

3. Why Both Metrics?

A model can have a low RMSE but poor ranking quality, or vice versa. For example, a model that always predicts the global mean (3.6) achieves a moderate RMSE but terrible MAP@10 since it cannot differentiate between items. Conversely, a model optimised purely for ranking may sacrifice rating accuracy. Reporting both metrics gives a complete picture of recommendation quality.

6. Experimental Results

Model	RMSE ↓	MAE ↓	MAP@10 ↑
SVD (Matrix Factorization)	0.9044	0.7049	0.0140
Item-Based CF	0.9588	0.7479	0.0090
ALS (Implicit)	1.8174	1.4461	0.0005

Table 3 — Model comparison on held-out test set (20% stratified split, 500 sampled users for MAP@10).

1. SVD Analysis

SVD achieves the best performance on both metrics — RMSE of 0.9044 and MAP@10 of 0.0140. This confirms the well-established strength of matrix factorization on the Netflix dataset. By learning compact latent representations that capture underlying taste dimensions (genre, era, style), SVD generalises far better than memory-based methods on sparse data. The bias terms (μ , b_u , b_i) also absorb user-level and item-level rating tendencies, reducing systematic errors.

2. Item-CF Analysis

Item-CF achieves a reasonable RMSE of 0.9588 — only 0.054 worse than SVD — suggesting it predicts individual ratings fairly well when similar items exist in the user's history. However, MAP@10 drops to 0.009, indicating weaker ranking quality. The cosine similarity matrix becomes noisy for items with few ratings, leading to suboptimal top-10 lists even when point predictions are adequate.

6.3 ALS Analysis

ALS underperforms on both metrics (RMSE 1.82, MAP@10 0.0005). This is expected: ALS was designed for implicit feedback (binary presence/absence), not explicit 1-5 star ratings. The confidence weighting ($c = 1 + 40r$) effectively amplifies high ratings to very large confidence values, distorting the objective and producing latent vectors that are poor predictors of the original rating scale. ALS would be far more appropriate if the task were to predict whether a user would watch a movie at all, rather than how highly they would rate it.

7. Recommendation Examples

1. User 250 — Action / Sci-Fi Enthusiast

This user's history includes The Mummy, Jaws, and Pirates of the Caribbean — adventure and action titles with strong franchise followings. SVD's top-10 list correctly identifies a preference for serialised, episodic content:

Rank	Title	Year	Pred. Score
1	Futurama: Monster Robot Maniac Fun Collection	1999	4.54
2	Stargate SG-1: Season 8	2004	4.52
3	Star Trek: Voyager: Season 5	1998	4.50
4	Star Trek: Voyager: Season 1	1995	4.45
5	The Sixth Sense	1999	4.44

Table 4 — Top-5 recommendations for User 250 (SVD).

Explanation: Recommended 'Futurama: Monster Robot Maniac Fun Collection' because you rated these highly: The Mummy, Jaws, Pirates of the Caribbean. These share an adventure / sci-fi flavour consistent with Futurama's tone.

7.2 User 1070 — Drama / Documentary Viewer

User 1070's history features Lord of the Rings, Pirates of the Caribbean, and Finding Nemo — a mix of epic and family titles. SVD surfaces prestige drama and quality serialised content, suggesting the model has captured a preference for high-quality, award-adjacent titles:

Rank	Title	Year	Pred. Score
1	Lost: Season 1	2004	4.83
2	Curb Your Enthusiasm: Season 3	2002	4.74
3	Foyle's War: Set 2	2003	4.71
4	The Life of Birds	1998	4.69
5	Samurai Champloo	2004	4.68

Table 5 — Top-5 recommendations for User 1070 (SVD).

7.3 Success and Failure Cases

	Observation
Success	Genre consistency — users with clear preferences (sci-fi, drama) receive thematically coherent list
Success	Era alignment — recommendations tend to match the decade of the user's rated titles.
Success	Serialised content — users who rated one season of a show often receive other seasons.
Failure	Cold-start users (< 20 ratings filtered out) cannot receive personalised recommendations.
Failure	Niche titles with few ratings score lower even if the user would enjoy them (popularity bias).
Failure	ALS recommendations are essentially random due to its poor MAP@10 (0.0005).

Table 6 — Success and failure case analysis.

8. Key Insights

Matrix Factorization dominates.

SVD consistently outperforms both memory-based (Item-CF) and implicit (ALS) approaches on explicit rating data. The ability to compress millions of sparse interactions into 100-dimensional latent vectors enables generalisation that neighbourhood methods simply cannot match.

Algorithm-data alignment matters more than model complexity.

ALS is a sophisticated model but is fundamentally misaligned with the task. Explicit ratings on a 1-5 scale are not implicit signals, and using confidence weighting designed for binary feedback produces poor results. Choosing the right model for the data type is more important than raw model power.

RMSE and MAP@10 tell different stories.

Item-CF achieves an RMSE close to SVD's (0.96 vs 0.90) but has significantly lower MAP@10 (0.009 vs 0.014). This highlights the difference between accurate point predictions and high-quality ranked lists — both matter in production systems.

Positivity bias affects the relevance threshold.

With a mean rating of 3.60 and median of 4.0, users rarely give low ratings. Setting the relevance threshold at 3.5 means roughly 62% of ratings are 'relevant', making MAP@10 a relatively lenient metric. A stricter threshold (e.g., 4.5) would give a more discriminating evaluation of top-tier recommendations.

Serialized content clusters naturally.

Multiple seasons of the same show appear consecutively in recommendations, suggesting SVD has learned a 'loyal viewer' latent dimension. This is a genuinely useful property for a streaming platform.

9. Future Improvements

Neural Collaborative Filtering (NCF):

Replace the dot-product interaction in SVD with a multi-layer perceptron. NCF can capture non-linear user-item interactions that linear factorization misses, particularly useful for users with eclectic taste profiles.

Hybrid Content + Collaborative Model:

Incorporate movie metadata (genre, director, cast) and user demographics to address the cold-start problem. New users could receive recommendations based on content similarity until enough interaction data is collected.

Temporal Dynamics:

The dataset includes rating timestamps. Models like TimeSVD++ exploit temporal drift — user tastes and item popularity change over time — and consistently outperformed static SVD in the original Netflix Prize.

Full Dataset Training:

We used 5% of users due to computational constraints. Training on all four combined_data files with a GPU-accelerated framework (PyTorch, JAX) would substantially improve model quality.

Hyperparameter Optimisation:

A systematic grid or Bayesian search over SVD's factors (50–300), regularisation (0.01–0.1), and learning rate would likely push RMSE below 0.87, approaching the benchmark set by the original Netflix Prize winners (~0.856).

Re-ranking and Diversity:

Post-processing the top-K list to balance relevance with diversity (avoiding ten seasons of the same show) and novelty (surfacing less-known titles the user would genuinely discover) would improve the practical user experience.